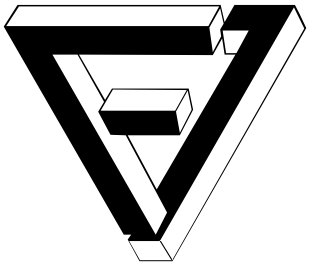
Masaryk Univerzity
Faculty of Informatics



Comparative Analysis of React, Vue.js, and Svelte: Technical Evaluation, Performance, and Developer Experience

Bachelor's Thesis

Martin Dékány

Brno, 2025

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source. During the preparation of this thesis, I used the following AI tools:

- Grammarly for grammar check,
- Github Copilot for faster code writing,
- ChatGPT to improve my writing style.

I declare that I used these tools in accordance with the principles of academic integrity. I checked the content and took full responsibility for it.

Advisor: RNDr. Marek Kumpošt, Ph.D
Consultant: Ing. Mgr. Jiří Šmerda

Acknowledgements

I would like to express my sincere gratitude to my advisor, RNDr. Marek Kumpošt, Ph.D., and my consultant, Ing. Mgr. Jiří Šmerda, for their guidance, insightful advice, and support throughout the development of this thesis.

Abstract

This thesis presents a comprehensive comparative analysis of three leading frontend frameworks, React, Vue.js, and Svelte, emphasizing technical evaluation, performance benchmarks, and developer experience. The research uses a uniform Vite-based build system to investigate each framework's architecture, project setup, state management, and rendering efficiency through a series of practical prototype implementations and performance tests. Quantitative benchmarks, including DOM update efficiency, state mutation in nested structures, event listener scalability, and input field synchronization, reveal significant performance and resource usage differences among the frameworks. The study also evaluates the ease of use, build size, and tooling ecosystems, highlighting that while React offers a rich ecosystem and stability, Vue.js provides a balanced approach with intuitive syntax, and Svelte excels in performance due to its compiler-based approach. The findings enable developers and architects to understand the trade-offs and strengths inherent in each framework. Based on the analysis, the thesis concludes with recommendations tailored to various project requirements, thereby offering a solid foundation for selecting an optimal frontend framework in modern web development projects.

Keywords

frontend frameworks, JavaScript frameworks, React, Vue.js, Svelte, comparative analysis, web development

Contents

Chapter 1

Introduction

In today's digital world, frontend frameworks play a crucial role in creating efficient, responsive, and engaging web applications. As demand for new, seamless, and dynamic web applications grows, developers rely more and more on frameworks such as React, Vue, or Svelte. Each of these frameworks offers a different approach to addressing challenges related to state management, rendering efficiency, and developer experience. In recent years, the increasing complexity of web applications and the need for high-performance solutions have increased the importance of selecting the proper framework for a project. These frameworks are currently one of the market's most popular and widely used frontend frameworks.

The purpose of this research is to provide a detailed technical evaluation of these frameworks, exploring areas such as performance benchmarking, state management approaches, and overall developer experience. The study is based on the hypothesis that each framework has its strengths and weaknesses and that the choice of framework can impact the development process and the final product. By using a consistent project setup powered by Vite and measuring performance metrics such as loading time, rendering speed, and memory usage, this research aims to provide an objective overview of the differences between these frameworks. These differences can guide developers in selecting the most suitable framework for their projects.

This thesis is organized into several sections. The first section provides an overview of the frameworks, including their history and key features. Followed by a detailed analysis of their performance, including benchmarks and comparisons. Then, an evaluation of the developer experience, including ease of use, learning curve, and community support, is discussed. Finally, the conclusion summarizes the findings and provides recommendations for developers and teams considering these frameworks for their projects.

1.1 Difficulty in choosing the proper framework

In the world of JavaScript, there is a significant movement at the moment. Things are changing fast, with new frameworks coming to the market now and then. Every time, there is a new methodology for developing software and explaining why one framework is better. Choosing from the dozens of frameworks available is crucial to choose the right one. Each framework has both advantages and disadvantages and what works for one project may not be suitable for another.

1.1.1 Framework selection

This thesis will only focus on the JavaScript frontend frameworks React, Vue, and Svelte due to their different approaches, popularity among developers, and strong community. While there are many other frameworks, the mentioned frameworks are among the most popular and widely used today.

Other frameworks, such as Angular or Ember.js, which also offer a different perspective on frontend development, will not be included in this study, in order to keep the content manageable and allow for a deeper exploration of the chosen frameworks.

1.2 Why use framework?

Modern web applications are becoming more complex and demanding, requiring developers to create efficient, responsive, and scalable applications. Frontend frameworks such as React, Vue, and Svelte have emerged as powerful tools to help developers meet these challenges, providing a set of tools and best practices to simplify the overall development process and improve code quality.

Additionally, frameworks offer an ecosystem with a wide range of libraries, tools, and community support, further enhancing the development experience and allowing developers to solve common problems more efficiently. They also solve cross-browser compatibility issues, ensuring that applications remain consistent across different platforms and devices [1].

With these advantages, frameworks simplify the process of building and maintaining complex applications and allow developers to focus on creating high-quality user experiences.

1.2.1 Components

Most modern frontend frameworks use component-based architecture. The component is a reusable piece of code, the same as a function. With a difference, that component also contains a small part of the UI (user interface)

with its logic [14]. Each component has its own state and props, which allows developers to create a more modular and maintainable application [10]. Simple component with props in React might look like this:

```
function BorderedButton(props) {
  return (
    <button style={{ borderColor: 'red' }}>
      { props.text ?? "Default value" }
    </button>
  );
}
```

Figure 1.1: Simple component with props in React.

This component, returns a button with red border and sets its text to given prop, or use default value. `BorderedButton` is easily called from another component, and its text can be changed as needed.

```
function App() {
  return (
    <main>
      <h1> Welcome to my app </h1>

      <BorderedButton />
      <BorderedButton text="Close" />
    </main>
  );
}
```

Figure 1.2: Using component in another component.

The main reason why components are important is that they allow developers to break down their problems into smaller sub-problems and then combine them to solve the problem efficiently. Breaking down an application into components helps with understanding the whole application as a whole, but also with reuse elsewhere in the application. Components act as foundational elements that, when assembled properly, construct the entire application.

Chapter 2

Overview

This chapter provides a brief overview of the history and evolution of frontend frameworks, as well as their key features and characteristics of React, Vue, and Svelte.

2.1 Evolution of Frontend Frameworks

In today's digital world, developing and deploying good-looking and interactive web applications is very easy. Anyone can watch a tutorial on YouTube and start developing their first web application in minutes. You can have a fully functional application with a few clicks and a few lines of code without much effort. However, it was not always like this. The history of web development is vibrant and engaging.

In the beginning, only static pages were used for information-sharing [2], [19]. They were not interactive, and they served as read-only documents. These documents were written in HTML and styled with CSS. Then, a new scripting language called JavaScript was introduced. With JavaScript, developers could create interactive and dynamic web pages.

However, as the web grew, so did the complexity of web applications, mainly browser compatibility issues. Developers had to put extra effort into ensuring their code was executed correctly on the different browsers. This raised many concerns and headaches, and much time was wasted on fixing these issues.

One of the first libraries that solved this issue was jQuery, which was first introduced in 2006. It contained many features from DOM (Document Object Model) selection and manipulation through event handling to AJAX (Asynchronous JavaScript and XML) requests. All of this with cross-browser compatibility [18]. It was a game-changer. Developers were able to write less code and focus on something that matters, functionality. Although jQuery has a lot of great utility functions, it lacks data management across multiple pages. This is where frontend frameworks come in.

In 2010, Google released AngularJS, a framework that addressed many challanges developers faced when building web applications. It introduced features such as two-way data binding, dependency injection, MVC (Model-View-Controller) architecture, and many more. While AngularJS was powerful, it was hard to learn with a steep learning curve. As projects grew, the codebase became more complex and harder to maintain, leading to frustration among developers [18].

In May 2013, Meta (formerly Facebook) released React. It had many features that AngularJS had, but it was simpler to learn and easier to maintain. React also introduced a new concept called Virtual DOM. Virtual DOM is a lightweight copy of the actual DOM that React uses to update the actual DOM [18]. This made React very fast and efficient. To this day, React is the most popular frontend framework, with over 34 million downloads per week as shown in figure 2.1.

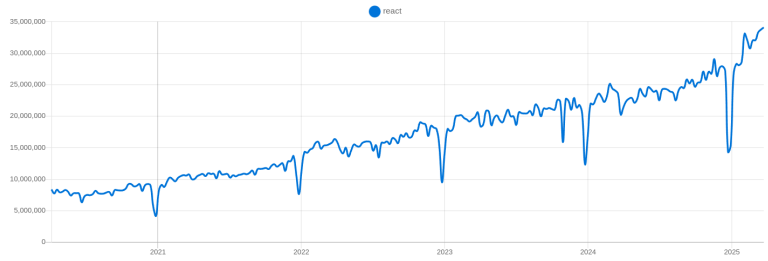


Figure 2.1: React weekly downloads. Adapted from [23].

After React, many new frameworks were released as the web development industry is constantly evolving.

2.2 React

React is maintained by Meta and a community of individual contributors and companies, which has gained popularity with its implementation of a concept called Virtual DOM [10].

This technique allows React to update user interfaces efficiently and, with that, increase the overall performance of the application. It achieves this by creating a virtual representation of the actual DOM, which reduces the number of direct manipulations needed by calculating differences in the actual DOM before and after a change. This way, React only updates nodes that were affected and need to be re-rendered [7], [13].

Virtual DOM is especially effective in applications where a lot of dynamic and interactive elements are needed to make user interface structure operations simpler and more effective than native methods [3].

React also uses a syntax extension for JavaScript called JSX, which allows developers to write HTML tags within JavaScript functions seamlessly. Although JSX and React are two different things, they are often used together [10].

This syntax extension allows regular developers to use React more naturally. Furthermore, JSX improves code readability and lets developers visualize the component structure better. Even if using JSX is optional, it has become a standard choice in most React projects due to its ease of use.

According to a survey [5] involving 14,015 people, React is the most used frontend framework, with 82% of respondents using it in the year 2024 as shown in figure 2.2.

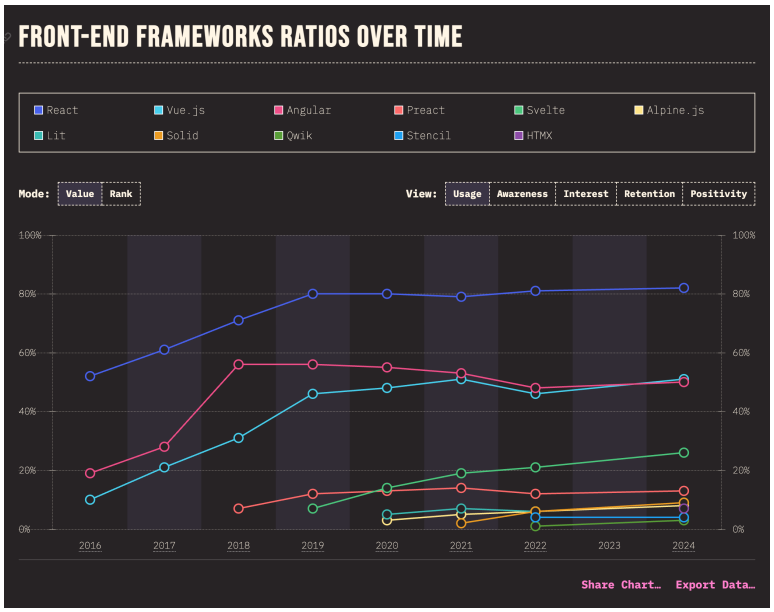


Figure 2.2: Usage of frontend frameworks ratios over time. Adapted from [4].

This is due to its strong community support and the many online resources that can help developers get started with React. Another source confirming React's popularity is a GitHub repository, where React has over 230k stars[1], being the most starred frontend framework. A more extensive survey that did not just focus on JavaScript and its technologies is a Stack Overflow Developer Survey, where React placed 1st among all web frameworks and technologies used by professional developers. Out of 38,132 respondents, 41.6% of them did extensive development work on React in the year 2024. Other can be seen in figure 2.3.

[1] https://github.com/facebook/react

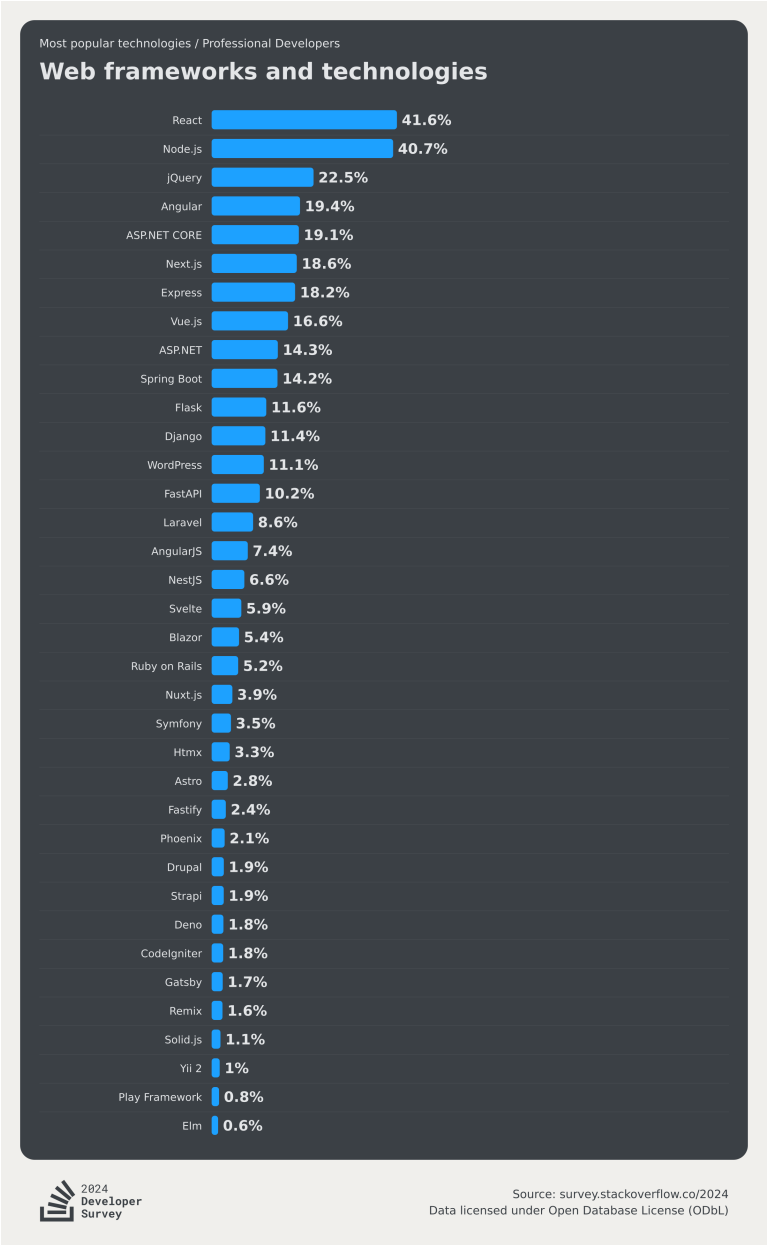


Figure 2.3: Stack Overflow Developer Survey 2024. Adapted from [12].

React's popularity can be due to its long-standing presence in the market and strong support from the community and big companies, making it a preferred choice for enterprise-scale projects. Additionally, React's huge community has created many open-source projects that can be used to extend React's functionality, creating a rich ecosystem around it. This simplifies the choice for developers, as they can find many resources and help online

when they encounter a problem.

Although React offers many features, it cannot work alone. To be fully functional, it needs additional third-party libraries, such as React Router or Redux. It is highly recommended to use another React-powered framework, so-called "Meta-frameworks," to extend React's functionality and make the application more efficient and maintainable [10]. Frameworks such as Next.js, Remix, Gatsby, or Expo (for native applications) are some of the most popular Meta-frameworks for React.

These frameworks solve common problems that developers face when building web applications, such as code-splitting, routing, search engine optimization, data fetching, and many more.

With the power of its frameworks, React gives developers much flexibility in rendering the user interface. Developers can choose between various rendering methods, such as server-side rendering, client-side rendering, hybrid rendering, static site generation, or native mobile applications. The choice of rendering method depends on the project requirements and the developer's preferences.

2.3 Vue

Vue is maintained by Evan You and a community of individual contributors. Vue is a progressive framework that can be incrementally adopted in existing projects without needing to rewrite the whole codebase [20]. It can also be used in new projects from the beginning. Vue uses HTML-based template syntax, compared to React's JSX syntax. For beginners, HTML-based templates are easier to learn, as the user interface is separated from the logic of components. Vue's templates are written in a single file that contains HTML, JavaScript, and CSS. This approach makes the code more readable and easier to maintain and provides separation of concerns. These templates are suitable for most projects and allow Vue's template compiler to apply optimizations to make applications faster. Even so, Vue allows developers to use render functions and JSX if needed [20].

Vue, like React, uses a virtual DOM as its rendering system. Unlike in React, Vue controls the compiler and the runtime, enabling the framework to implement many compile-time optimizations. The compiler performs static analysis on the template and writes hints in the generated code to allow the runtime to take shortcuts whenever possible. This approach is called "Compiler-Informed Virtual DOM" [20]. This makes Vue faster and more efficient, as it can skip unnecessary checks and updates. A few examples of optimizations that Vue can make are:

- **Static Hoisting** - skip diffing static parts of the template

- **Patch flag** - optimize updates for elements with dynamic bindings
- **Tree Flattening** - greatly reduces the number of nodes that need to be checked

Since Vue is a community-driven project [20], it offers a rich ecosystem of plugins that can be used to extend its functionality. The most popular plugins include Vue-Router, Vuex, Vuetify, and many more.

Like React, Vue also offers Meta-frameworks that can be used to extend Vue's functionality and make applications more efficient and maintainable. Some of the most popular Meta-frameworks are Nuxt.js, Quasar, and VitePress.

Vue developers can also choose between different rendering methods, such as server-side rendering, client-side rendering, static site generation, or mobile applications.

2.4 Svelte

Svelte is maintained by Rich Harris and a community of individual contributors and is backed by Vercel. Among React and Vue, Svelte is the newest framework, gaining the most excitement among developers.

Svete is trying to solve the same problems as React and Vue but in a different way. Unlike React and Vue, Svelte does not rely on a runtime library that manages the state and updates the DOM. Instead of using virtual DOM, Svelte compiles your code into efficient pure JavaScript at build time and updates the DOM with highly optimized code. This makes Svelte fast and more efficient, as it does not have to manage and compare virtual DOM and calculate the minimal set of changes that need to be made to the actual DOM. Thus, Svelte is more like a compiler than an actual framework. It takes code and compiles it into a highly efficient plain JavaScript code. Svelte uses a concept called "runes" to control the Svelte compiler, which is part of the language itself [22]. It is a Svelte way of telling the framework, what is dynamic, and what is static in the component.

Svelte uses HTML-based template syntax, similar to Vue. Markup inside a Svelte component is written in a single file, making it easier to read. Svelte's templating language has a special directives, such as {#if}, {#each}, {#await}, etc., that are used to render the content dynamically [22].

Since Svelte is quite a new framework, it has a smaller community than React and Vue. Although Svelte has a smaller community and ecosystem, it is growing quickly. Based on the State of JavaScript survey, Svelte has been the most interesting framework for developers 6 years in a row since 2019 as can be seen in figure 2.4.

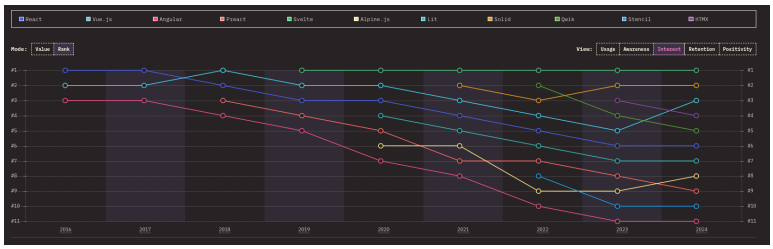


Figure 2.4: Frontend frameworks interest over time. Adapted from [5].

This is might be due to its simplicity and ease of use. Stack Overflow Developer Survey also confirms Svelte's positive impact on developers, with 72.8% of respondents who used Svelte wanting to continue using it in future [12].

Like React and Vue, Svelte offers a Meta-framework that can be used to extend Svelte's functionality and make applications more efficient and maintainable. The official and most popular Meta-framework for Svelte is SvelteKit, which offers features similar to Next.js for React or Nuxt.js for Vue.

Chapter 3

Deep Dive Comparison

The goal of this chapter is to develop one performance-focused and one sample application. These applications will be used to compare the performance and ease of use of different frameworks. Meta-frameworks, which offer many advantages and solve many problems, will not be implemented due to the scope of this thesis, but will be mentioned.

3.1 Project Setup

For the initialization of the project, Vite is used as a build tool for all applications to ensure a simple and uniform project setup. Vite is chosen due to its faster development experience and smaller bundle size compared to other build tools such as Webpack [17], [11]. Another important note is that Node.js version 22, and npm version 10, are used to develop the applications. Every application is created with the `npm create vite@latest` command. Choosing the framework is done by selecting the desired framework from the list of available options. Continuing with the default options, the application is then created. The application uses TypeScript to ensure type safety and better code quality. Setup for all three applications is the same, with the only difference being the selected framework. Therefore, the initialization is straightforward.

3.2 Methodology

All applications were developed using the same components and features. Everything is implemented based on the official documentation of each framework, to ensure that the comparison can reflect the typical experience of a developer starting with a given framework, without prior knowledge or optimizations. This ensures that the comparison reflects how an average developer might experience each framework when following official guidelines, rather than relying on community-driven best practices or optimizations.

The hardware used for the performance test is a HP Omen 15 laptop with the following specifications:

- Processor: AMD® Ryzen 7 5800H
- RAM: 16GB DDR4
- Storage: 512GB SSD
- Operating System: Pop!_OS 22.04 LTS

All applications were tested on the same hardware in the production mode to ensure comparable results.

3.3 Performance

The prototype for the performance test is an application divided into four parts, as seen in the figure 3.1. Each part is a separate component, which is then rendered on the screen. The application is designed to test the rendering speed of the framework under heavy DOM manipulation. Google Chrome's performance tool and Performance API measure the time it takes for the application to render changes on the screen.

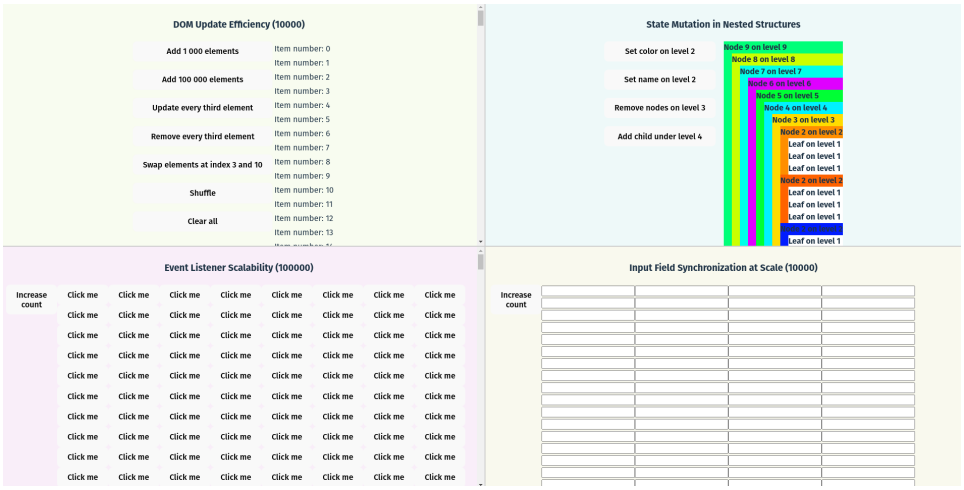


Figure 3.1: Screenshot of the performance test prototype application.

Firstly, there is a difference in state management between React and Vue with Svelte. The state in React should be treated as immutable [10]; changes are made by creating a new state object and calling React hooks. Vue and Svelte, on the other hand, allow the direct mutation of the state object. This is due to the Proxy object in JavaScript, which allows a deeply reactive state [22].

Figure 3.2 and figure 3.3 shows the difference between immutable and mutable state in React and Svelte.

```
const [items, setItems] = useState([1, 2]);
setCount(prevItems => [...prevItems, 3]);
```

Figure 3.2: Example of adding an item to a immutable array in React.

```
let items = $state([1, 2]);
items.push(3);
```

Figure 3.3: Example of adding an item to a mutable array in Svelte.

This difference in state management can have a significant impact on the performance of the application. React's immutability can lead to a lot of unnecessary re-renders, as entire components are re-rendered when the state changes. Vue and Svelte's approach is more intuitive, as developers can use common JavaScript operations to mutate the state directly.

In order to make an accurate comparison, state management in all three applications must be implemented in the same way. The state is stored in the root component and passed down to the child components as props. State changes are then passed back to the root component through direct state mutation. In the case of React, the third-party library called `use-immer` is used to allow direct state mutation. This is a recommended way to handle deep state changes in React applications [10]. Under the hood, `use-immer` uses the Proxy object to allow direct state mutation to write convenient syntax [10]. However, it still produces copies of the state object and acts more like syntactic sugar for the developer.

3.3.1 DOM Update Efficiency

The first part of the test measures the total time it takes for the application to expand a table with 10,000 records by another 100,000. The table is rendered using a `<table>` element and `<tbody>`. Each row is rendered using a `<tr>` element and `<td>` elements for each cell.
Measurements focus on the time spent on:

- Scripting
- Rendering
- Painting

Scripting time is the time to execute JavaScript code, rendering time contains building the DOM tree and calculating styles, and painting time is the process of drawing pixels on the screen.

The function to expand the table by 100,000 records is the same for all three applications and is shown in figure 3.4.

```
function addElements(arr: unknown[], n: number) {
  const startLength = arr.length
  arr.length += n

  for (let i = startLength; i < startLength + n; i++) {
    arr[i] = i + 1
  }
}
```

Figure 3.4: Function to expand the table by 100,000 records.

The performance test results are shown in the figure 3.5.

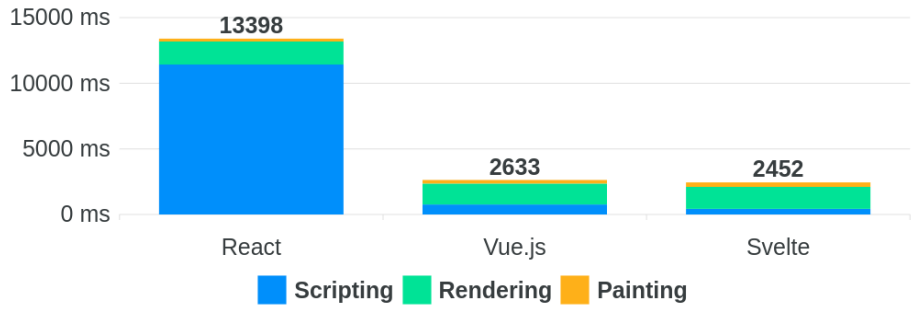


Figure 3.5: Expansion of the table with 10,000 records by another 100,000.

For the first test, React is the slowest framework, averaging 13.4s to render the table. Most of the time is spent on scripting due to React's immutability and re-rendering of the entire component tree. Vue and Svelte are much faster and are similar in terms of performance. If we look at JavaScript heap usage, we can see a different point of view on a given issue.

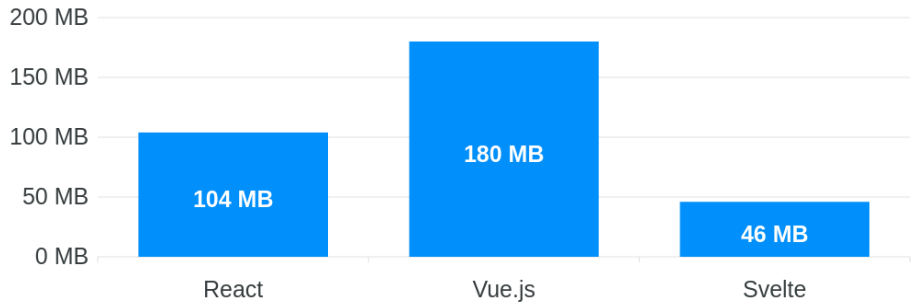


Figure 3.6: JavaScript heap usage increased during the expansion of the table by 100,000 records.

Interestingly, Vue uses 1.8x more memory than React and almost 4x more than Svelte. This might be due to its optimization of the reactivity system, which is more complex than React's or Svelte's. Svelte uses the least amount of memory, which is expected due to its compiler-based approach.

Other measurements are obtained using the `requestAnimationFrame()` function. This function schedules a callback to execute just before the next repaint [6], which allows us to measure the time it takes for the application to render changes on the screen. Additionally, I utilized `performance.now()`, a high-resolution timer offering sub-millisecond precision [6], making it ideal for benchmarking and performance analysis. Every operation is performed 10 times, and the average time is calculated on the table with 100,000 records. See the table 3.1 for the results.

	React	**Vue.js**	**Svelte**
`Update every 3rd row`	2,72s	1,82s	1,80s
`Swap 2 rows`	0,32s	0,15s	0,22s
`Shuffle all rows`	28,75s	2,77s	2,61s
`Delete every 3rd row`	0,92s	1,34s	0,65s
`Clear all`	0,39s	0,29s	0,11s

Table 3.1: Average time to perform operations on the table.

Svelte is the clear winner when it comes to performance in DOM manipulation. It is the fastest framework in all performance tests and uses the least memory. React is noticeably slower when adding or reordering keyed elements in the DOM. Vue is slightly slower than Svelte, but in general is heavier on the memory usage with large data sets.

3.3.2 State Mutation in Nested Structures

Although I used `use-immer` to allow direct state mutation in React, which is the recommended way to handle deep state changes, it is an additional dependency that needs to be installed. The second part measures the time it takes to render changes in deeply nested objects without using `use-immer`. This shows how React is used without additional libraries and how it is compared to Vue and Svelte. Firstly, a tree with a height of nine is created, where each parent has three children, resulting in 9841 nodes. In mathematical terms, the number of nodes in a tree with height h and branching factor b is given by the formula:

$$\sum_{i=0}^{h-1} b^i$$

From the results in figure 3.7, we can see that Svelte is again the fastest framework in all four operations. React beats Vue in three out of four operations and is only slower in the `Add child under level 4` operation, which confirms that React struggles with adding new elements to the DOM, as seen in expansion of table 3.5.

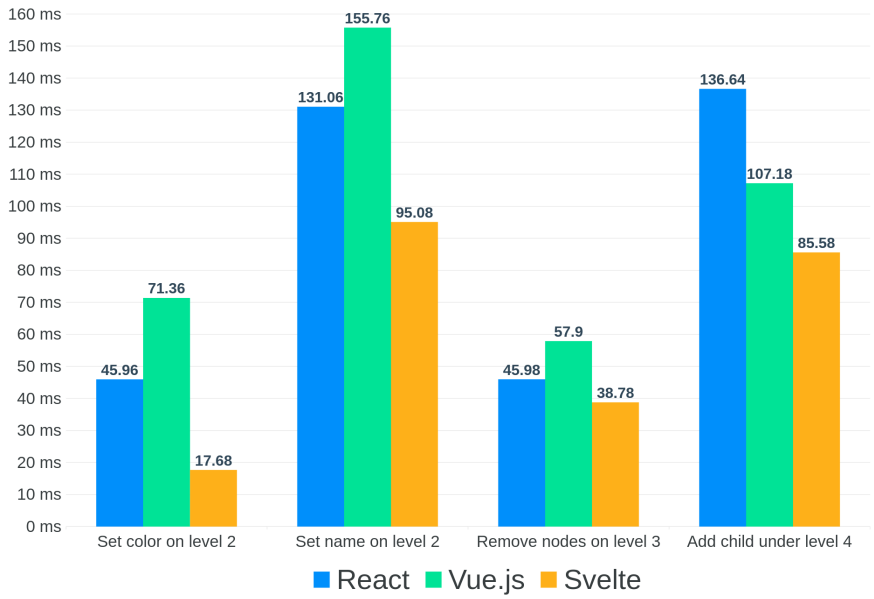


Figure 3.7: Deeply nested tree operations duration.

The disadvantage of React regarding developer experience is its need to create a new state object when updating the state. This can lead to more complex code requiring a different state management approach than Vue or Svelte. Example of adding a child under a certain level with reference to the tree in React:

```
  const addNodeOnLevel = (level: number) => {
    const rec = (currentNode: Node): Node => {
      if (currentNode.level === level) {
        return {
          ...currentNode,
          children: [
            ...currentNode.children,
            generateTree(level - 1)
          ],
        }
      }

      return {
        ...currentNode,
        children: currentNode.children.map(rec),
      }
    }

    // React's function from useState hook
    setTree((prevTree) => rec(prevTree))
}
```

Figure 3.8: Example of adding a child under a certain level with reference to the tree in React.

Example of adding a child under a certain level with reference to the tree in Vue and Svelte:

```
  const addNodeOnLevel = (level: number) => {
    const rec = (currentNode: Node, level: number) => {
      if (currentNode.level === level) {
        currentNode.children.push(generateTree(level - 1))
      }

      currentNode.children.forEach((child) => rec(child, level))
    }

    rec(tree, level)
  }
```

Figure 3.9: Example of adding a child under a certain level with reference to the tree in Vue and Svelte.

Vue and Svelte are more intuitive, allowing direct state mutation, which

aligns more with common JavaScript practices.

Performance issues with deeply nested objects could be further optimized by normalizing and flattening the state.

3.3.3 Event Listener Scalability

The third part of the test measures the count of event listeners when increasing the count of buttons. Each button has an event listener attached to it, with a simple `console.log()` function. Findings from Vue.js and React are the same. The event listener is attached to the new button when adding a new button. This is expected behavior, as each element has its own event listener. Meaning that the count of event listeners is equal to the count of buttons. Svelte, on the other hand, has only one event listener for all buttons. This is due to the way Svelte handles event listeners. Instead of attaching an event listener to each element, Svelte attaches a single event listener at the application root to handle events [22]. This technique, called event delegation, improves performance and reduces memory usage. Event delegation can also be used in React and Vue, but it is not the default behavior and requires additional knowledge and code to be written. In Svelte, this is done automatically to most commonly used events, such as `click`, `change`, `mouseover`, etc. By making it the default behavior, Svelte can reduce the count of event listeners, making the same elements to have only one event listener.

This is an excellent example of how Svelte can optimize the application's performance without additional code.

3.3.4 Input Field Synchronization at Scale

The last part of the test measures the time it takes to synchronize large amounts of input fields. This can be useful when working with applications that use many input fields, e.g., spreadsheets, forms, etc. Measurements are done on 10,000 input fields, where each input field is synchronized with the state. The word length of each input is set to 5, 10, and 15 characters to see how the length of the input affects the performance. Results are shown in the figure 3.10.

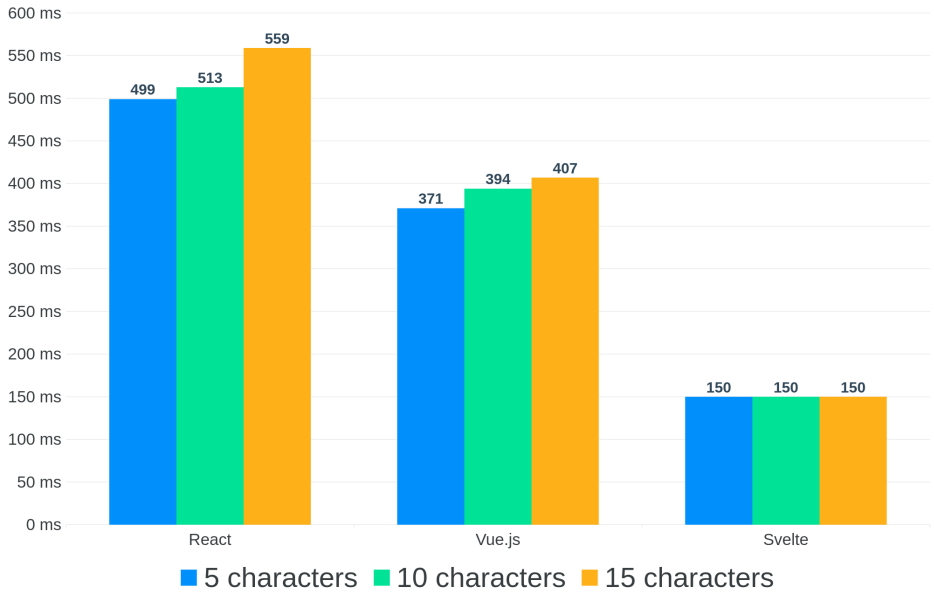


Figure 3.10: Synchronization of 10,000 input fields.

Durations are increasing with the length of the input, except for Svelte, where the duration is the same. This result may differ if different word length is used for the input fields, but it is clear that Svelte is the fastest framework in this test. In real-world applications, inputs commonly do not share the same state but are rather independent from each other. However, it is still interesting to see this performance difference between frameworks.

3.4 Ease of use

The second application is a simple to-do list application, which is used to test the ease of use of the framework and overall developer experience. A to-do list application is a typical example used to demonstrate a framework in action, as it is simple to understand and implement. The application consists of a list of tasks, where each task can be marked as completed or deleted. These tasks are fetched from external API and stored in a global state. Other to-do items can be added to the list by entering the task name in the input field and clicking the add button. On top of that, animations are added to the application to make it more visually appealing and demonstrate how animations are commonly used. The application is shown in the figure 3.11.

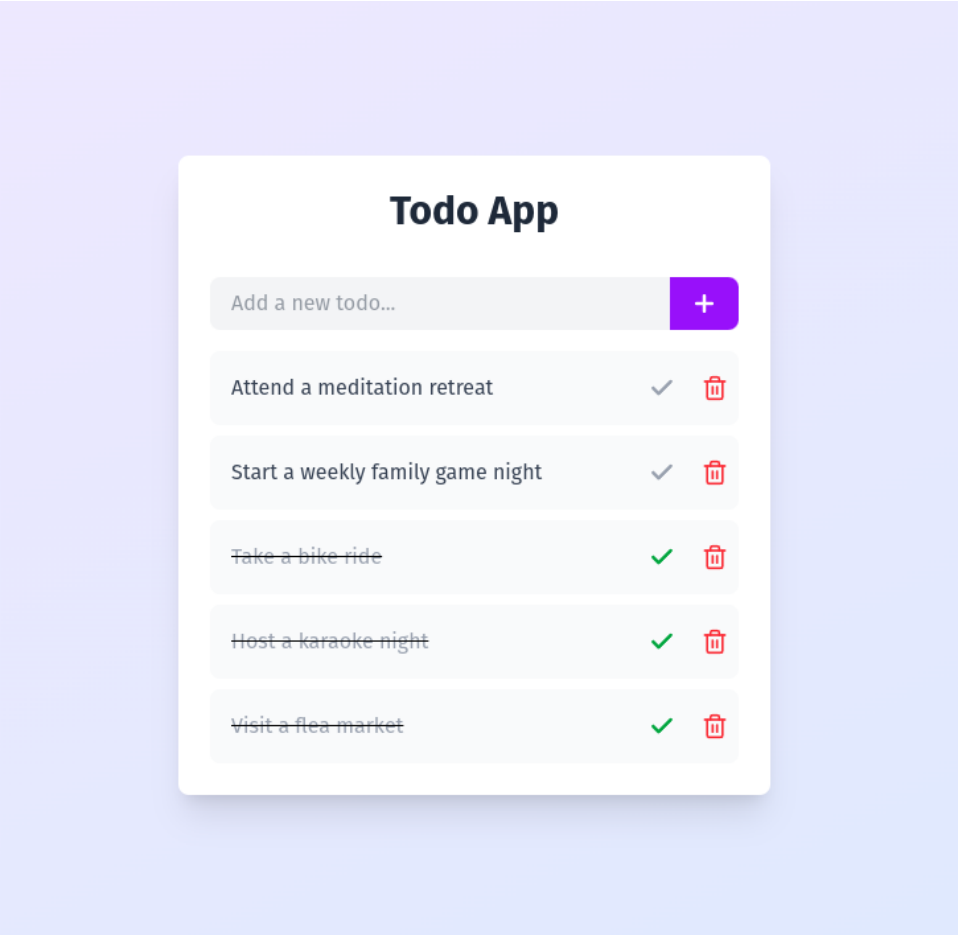


Figure 3.11: Screenshot of the sample application.

3.4.1 Project

Setting up the project is the first step in the development process. This step should be simple, to allow developers to start working on the application as soon as possible. Initialization of each framework is done the same way as in the project setup section 3.1. Therefore, this step for all three frameworks is effortless and straightforward. TypeScript is set up without the need for additional configuration, and then the application is ready to be developed.

3.4.2 Tailwind CSS

Tailwind CSS is a utility-first CSS framework [16] used to style the application. This new styling approach allows developers to rapidly build designs without writing custom CSS. Tailwind CSS differs from traditional CSS frameworks, as it does not provide pre-designed components but rather pro-

vides utility classes that can be easily added to the HTML elements. Based on the ranking from the research project, Tailwind CSS is overall the best CSS framework, with the highest score [15].

The installation of Tailwind CSS is straightforward because Vite is used as a build tool. This step is also the same for all three frameworks.

3.4.3 Components

Components are the building blocks of the application and should be easy to create and use. In Vue and Svelte, components are created by creating a new file with a `.vue` or `.svelte` extension. React components are commonly created by creating a new file with a `.tsx` extension and exporting a function. The application imports and uses components by adding the component tag to the template.

Components in Vue and Svelte are easier to read and understand, as JavaScript and HTML are separated into different blocks. Additionally, Vue and Svelte support CSS in the same file, whereas React requires an additional file for styling. Another great feature is Scoped CSS, which is supported by all three frameworks, allowing one to style the component without affecting other components, even if the same class names are used.

When it comes to passing data from parent to child components, props are used. In Vue and Svelte, props are defined using a special function of each framework, for example, `defineProps<Todo>()` in Vue. This function is used to define the type of the prop and provide default values. In React, props are defined as parameters of the exporting function. React's approach is more intuitive for newcomers to the framework, as it is similar to regular JavaScript functions and does not require additional knowledge of the framework. Direct mutation of the props is not allowed by default in all three frameworks; however, in Vue and Svelte, there is a way to mutate the props by using special functions provided by the framework. In React, mutating the props is not possible.

Overall, components in Vue and Svelte are easier to create and use due to the separation of JavaScript and HTML and support for CSS in the same file.

3.4.4 Global State

To demonstrate global state management, the application uses the global state to store the list of tasks and to perform manipulations on the list. The global state shares data between components without passing props down the component tree, which is known as `prop drilling`.

Global states are supported differently in each framework. In React, the global state can be managed using Context API, which is a built-in feature of React, but this approach does not scale well and can lead to performance

issues [8]. Third-party libraries such as Redux or Zustand can be used to solve this issue. In Vue, the global state can be managed using Pinia, which is maintained by the Vue core team [20]. In Svelte, the global state is managed using stores, which are built-in features of Svelte [22]. The advantage of Vue and Svelte is that only one store is supported by documentation, which is beneficial for developers, as they do not need to spend time choosing the right library. In this application, a global state in React is managed using Zustand, which is a fast and small library [21].

Vue's Pinia is easy to use, as it uses the same functions for state management as Vue's reactive state. There is no need to learn additional concepts; define store with `defineStore()` function and use it in the application. Svelte's store is also easy to use but requires additional knowledge of the framework, as it uses different functions and has different approaches to updating and setting the state. A few functions are available, but they are enough to manage and subscribe to the global state. This additional knowledge can be picked up quickly. Lastly, React's global state management is complex to compare because it relies on third-party libraries, which can have different approaches to state management and require additional knowledge of the library. In this application, Zustand is used, which is easy to use, and React's concept of hooks is used for deriving and updating the state. For React developers, Zustand is a great choice, as it relies on React's concepts and uses minimal boilerplate code that can be easily understood. One disadvantage of Zustand's store is that the type definition is not automatically inferred and needs to be defined manually.

3.4.5 Animations

Animations are included in the application to demonstrate how each framework handles this aspect of UI development. While animations are not something frameworks commonly provide; they are an important part of the application, as they can make the application more visually appealing. Each framework can be created purely using CSS or JavaScript, but some libraries can be used to simplify the process. In React, developers must use third-party libraries such as Framer Motion or React Spring to create animations. This further complicates the development process, as developers must learn how to use and integrate the library with the application. Vue, on the other hand, provides built-in support for animations, which makes it easier to create animations. In Vue, a special `<Transition>` or `<TransitionGroup>` component wraps the elements that should be animated when added or removed from the DOM or provides documentation on other ways to animate elements. This way, the animation process is simplified, but developers must still develop animations independently. Svelte, like Vue, provides built-in support for animations but with more features. Svelte provides `transition`, `animate`, `in`, and `out` directives, which can be

used to animate elements. Svelte also provides a list of predefined animations that can be extended and customized to fit the application. Everything is built-in, and there is no need for additional libraries.

3.4.6 Build

Build size might not be the most important factor when choosing a framework, but it can still be a deciding factor regarding user experience. Smaller build size means faster loading times, reduced bandwidth usage, and better performance on low-end devices or slow network connections. This is particularly important for users in areas with limited connectivity or those using mobile data. Faster load times improve user experience and can lead to better SEO ranking, as Google uses page speed as a ranking factor [9]. In this application, the build is created using Vite and compressed using gzip or brotli. Each build consists of the index.html file, document icon, and generated JavaScript and CSS file. The sum of the sizes of all files is used to calculate the build size. Results are shown in the table 3.2.

	React	**Vue.js**	**Svelte**
`without compression`	299.2 kB	170.3 kB	71.8 kB
`gzip`	98.9 kB	63.8 kB	28.3 kB
`brotli`	87.0 kB	55.4 kB	25.1 kB

Table 3.2: Total build size of the application.

Svelte is the smallest framework in terms of build size, with the smallest size of the generated files. Vue's size is in the middle, and React is the largest framework in terms of build size. The compressed build size is significantly smaller than the uncompressed build size. From the results, it is clear that the production version of the application should be compressed to reduce the build size and improve user experience.

Build time is another important factor, as it can affect the development process. Compression of the production build had a negligible impact on the build time and is not included in the results. Results are shown in the table 3.3.

	React	**Vue.js**	**Svelte**
`development`	0.33s	0.38s	0.48s
`production`	2.29s	2.18s	4.60s

Table 3.3: Average build time of the application.

In build time, React is the fastest framework. Svelte, being the slowest, is caused by the additional optimizations that Svelte does to improve the application's performance.

Although the measurement was performed on a prototype application, real-life results may vary significantly as new libraries, components, and features are added to the application. In general, Svelte is the smallest framework in terms of build size and the slowest in terms of build time, On the other hand, React is the largest framework in terms of build size and has the fastest build time.

3.5 Developer Experience

This section will focus on each framework's overall DX (Developer Experience). DX is a term used to describe how developers are satisfied with the framework and how easy it is to develop applications. Documentation, ease of learning, tooling, community, and ecosystem are considered factors.

3.5.1 Documentation

For most developers, documentation is the first place to look for when starting with a new framework, as it provides information about the framework, how to use it, and what features it offers. Documentation should be easy to read and understand and should provide examples and code snippets to help developers quickly understand the concepts.

React's documentation is well-written, pleasant to read, and provides many examples and code snippets. In a section called "Quick Start," developers can find tutorial on creating a simple Tic-Tac-Toe application that covers only a few basic concepts. In their documentation, many code snippets are editable, which allows developers to experiment with the code and see the results in real time. This way, documentation is more interactive, and developers can learn by doing. React also covers a lot of pitfalls and common mistakes, which can help developers to avoid them and improve the quality of the code. Overall, React's documentation is great, covers a broad range of topics, and is easy to read and understand.

Vue's documentation is also well-written and provides many examples and code snippets. These snippets are not directly editable but often link to their playground, where developers can experiment with the code. In the first section, called "Introduction," developers can find three different paths to learn Vue, which is great for developers with different levels of experience or different learning styles. They can choose from trying tutorials, reading the guide, or checking examples. This tutorial covers more topics than React's tutorial and is more in-depth, focusing more on the core concepts of Vue. The tutorial is more interactive and easier to follow. Vue's documentation is easier to navigate, as all topics are visible by default on the left side of the screen. Vue's documentation is interactive and provides many examples developers can easily understand. A great feature of Vue's documentation

is that it provides many examples and has built-in playground, where developers can experiment with the code and see the results in real-time.

Svelte's documentation, compared to React and Vue, is written worse. It is harder to read and understand and provides fewer examples and code snippets. These snippets are not editable, although a demo is provided; similar to Vue's playground, this demo link is hard to find and only provides a few examples. On the good side, Svelte provides a deep dive tutorial covering a wide range of topics. This tutorial is much more in-depth compared to React's and Vue's tutorials, and also covers more advanced topics. Even though Svelte's documentation is the worst among the three frameworks, they aim to get new developers familiar with Svelte through the interactive tutorial that walks them through each framework feature, after which they can fully leverage the power of Svelte. Tutorial's interactivity is also enforced by the fact that copy and paste is turned off by default, and developers must to type the code themselves, which can help them remember the concepts better. Most of the information about the framework is inside the tutorial, which can be harder to navigate. However, this form of documentation might be a new standard that other frameworks will gradually adopt.

3.5.2 Ease of learning

When starting with a new framework, developers should be able to learn the basics quickly and start developing applications. This can be achieved by having good documentation and a lot of examples and code snippets. The framework should be easy to understand and easy to use.

In React, developers should learn how to use JSX, which can initially feel unintuitive, especially for those from an HTML and JavaScript background. Additionally, React's approach to state management with concepts such as immutability and hooks can be overwhelming for new developers. React has extensive documentation, as mentioned in the subsection 3.5.1, and a large community will be discussed in the subsection 3.5.4, with many resources available online. Beginners may find its ecosystem overwhelming due to the necessity of additional libraries, which offer many options. Making the learning curve steeper.

Conversely, Vue offers a smoother learning curve and is designed to be easy to understand and use. Its template syntax is similar to HTML, which makes it easier for new developers to get started, with no need to learn new syntax. Vue's reactivity system is more intuitive, allowing developers to directly mutate the state, which aligns more with common JavaScript practices. Vue's documentation is also well-written and provides many examples and code snippets, which can help developers learn the framework quickly.

Svelte's syntax is similar to Vue's, focusing mainly on simplicity and ease of use. Svelte's reactivity system is also more intuitive, allowing developers

to mutate the state like Vue directly. Its hands-on approach to learning is great for new developers, as it guides them through the process of learning the framework. Svelte's documentation is not as extensive as React's or Vue's. Also, Svelte's ecosystem is smaller, making it harder for developers to find resources and libraries. However, Svelte's simplicity and ease of use can make it easier for developers to learn the framework quickly.

Example of creating a simple component in React:

```
import { useState } from 'react'

function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  )
}
```

Figure 3.12: Example of creating a simple component in React.

Example of creating a simple component in Vue:

```vue
<template>
  <div>
    <p>You clicked {{ count }} times</p>
    <button v-on:click="increment">Click me</button>
  </div>
</template>

<script setup lang="ts">
  import { ref } from 'vue'

  const count = ref(0)

  function increment() {
    count.value++
  }
</script>
```

Figure 3.13: Example of creating a simple component in Vue.

Example of creating a simple component in Svelte:

```svelte
<script lang="ts">
  let count = 0

  function increment() {
    count++
  }
</script>

<p>You clicked {count} times</p>
<button on:click={increment}>Click me</button>
```

Figure 3.14: Example of creating a simple component in Svelte.

The examples above show that Vue and Svelte are easier to read and understand as they separate JavaScript and HTML. In React, JSX is used to write HTML in JavaScript, compared to Vue and Svelte, where HTML is written in separate blocks.

Overall, Vue and Svelte are easier to learn compared to React due to their simpler syntax and reactivity system. However, React's extensive resources and community support can make the gap smaller.

3.5.3 Tooling & Ecosystem

The tooling and ecosystem around the framework significantly affect the DX and developer productivity. Each framework has its own set of tools and libraries that can help developers build and deploy applications.

DevTools DevTools can help developers debug and provide insights into the overall structure of the application as well as performance inside the browser. Vue's dev tools are well-looking and easy to use. They provide many features, such as a component tree, editable props, support for pinia, or the ability to open components in the editor by clicking on an element in the browser. An interesting feature is a component graph, which shows the relationship between components and can help developers further debug the application and decouple the components. The graph is shown in the following figure 3.15.

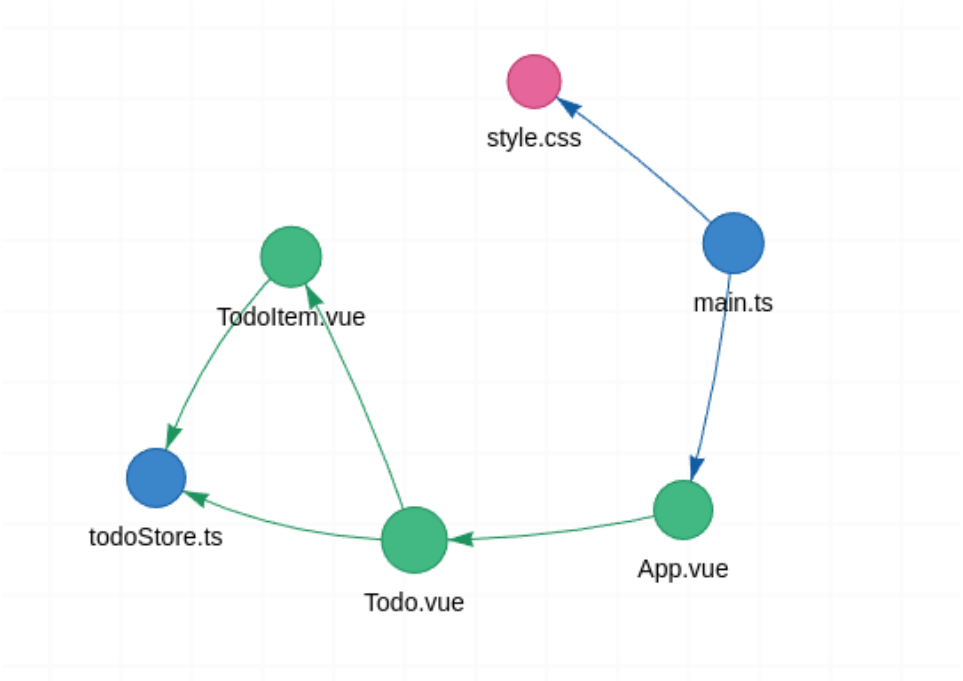


Figure 3.15: Vue DevTools component graph.

React dev tools also support the overall structure of the application in the component tree and editable props. They do not provide as many features as Vue's dev tools, mainly focuses on performance, which may be more important than additional features. Regarding Svelte, dev tools are unavailable for Svelte 5, the newest version of Svelte. This is a considerable disadvantage, and therefore, Svelte dev tools are not described in this comparison.

Meta-frameworks Each framework offers its own meta-framework, which can further improve the development process. Besides providing additional features or structure to the codebase, these frameworks can also support different rendering modes. Rendering modes are important, as they can affect the application's performance, which can be crucial for the user experience.

Some of the most used rendering modes are:

- Client-side rendering (CSR) - rendering the application on the client and fetching the data from the server
- Server-side rendering (SSR) - rendering the application on the server and sending the pre-rendered HTML to the client
- Static site generation (SSG) - generating static HTML files at build time and serving them to the client
- Progressive web application (PWA) - an application that can be installed on the device and works offline
- Island architecture - a combination of CSR and SSR, where some parts of the application are rendered on the server and some on the client
- Native mobile application - an application that can be built for iOS and Android, using the same codebase

React has many options to choose from, such as Next.js, Remix, React Native, and many more. Next.js is the most popular meta-framework for React and provides many features, such as built-in optimizations, routing, and server-side rendering. Next.js is widely used by developers and recommended by the React team. There are many options, with plenty of features, and great documentation and community support. Using the React meta-framework, developers can utilize each of the rendering modes. When it comes to mobile development, React is gaining a lot of popularity.

Vue has fewer options to choose from compared to React, frameworks such as Nuxt.js or Quasar. Nuxt.js is the most popular meta-framework for Vue and provides features similar to Next.js, such as built-in optimizations, routing, and server-side rendering, which support most of the rendering modes. Quasar is more focused on building cross-platform applications and provides support for building applications for iOS, Android, and the web using the same codebase. For mobile development, Cordova or Capacitor can be used, which uses WebView to render the application and provide access to the device's features. Compared to React Native, these applications are not as performant, as they do not use native components.

Lastly, Svelte has only one actively maintained meta framework called SvelteKit, which is officially supported by the Svelte team. SvelteKit is similar to Next.js and Nuxt.js, which have similar features and support common rendering modes.

User Interface libraries When it comes to the actual building of the application, developers often use UI (User Interface) libraries, which provide pre-designed components that can be easily added and customized to the application. UI libraries can help developers build applications faster and improve the application's overall design. React has a rich ecosystem of UI libraries, such as Material UI, Ant Design, Chakra UI, or shadcn-ui. Each has tens of thousands of stars on GitHub and is widely used by developers. This list is not exhaustive, and many more UI libraries are available for React.

Vue also has a rich ecosystem of UI libraries, such as Vuetify, Buefy, Element Plus, and many more. These libraries are mature and provide many components. Compared to React's UI libraries, there are fewer to choose from, but still enough to build a great application.

Lastly, Svelte has the smallest ecosystem of UI libraries, and among the most popular are Svelte Material UI, Melt UI, and Flowbite Svelte. These libraries are not as mature as React's or Vue's libraries.

3.5.4 Community

Another factor to consider is the community around the framework. Community is an important part of the framework, as it provides support, and help developers solve problems. The community can also provide additional resources, such as libraries, tools, and tutorials, that can help developers improve their skills and develop better applications. Metrics that can be used to measure how big and active the community are:

- Number of stars on GitHub - shows the overall popularity of the framework
- Number of contributors - shows how many people are contributing to the framework
- Number of issues and pull requests - shows how active the community is
- Number of npm downloads - shows how widely the framework is used
- Number of questions on Stack Overflow - shows how many people are asking questions about the framework

Measurements are taken from GitHub, npm, and Stack Overflow and are shown in table 3.4. Vue 2 (an old, not maintained version of Vue.js) is added to the comparison related to GitHub, as it has a different repository than a new version of Vue.js. GitHub stars, contributors, and issues from these two repositories cannot be added together, as there are some duplicates, and it would not be an accurate representation of the community.

	React	Vue.js (+Vue 2)	Svelte
GitHub stars	232k	49k (+208k)	81k
contributors	1 688	529 (+366)	762
issues (open)	772	645 (+357)	766
issues (closed)	12 777	4 891 (+9 665)	7 227
pull requests (open)	178	349 (+247)	50
pull requests (closed)	16 719	4 783 (+2 317)	6 673
npm downloads (last 7 days)	31.8m	6.4m	2.2m
Stack Overflow questions	479k	108k	6k

Table 3.4: Community metrics of the frameworks.

The results show that React is the most widely adopted and actively developed framework, with the most significant community and npm downloads. While Vue has a big community, it is not as big as React's. On the other hand, Svelte has the smallest community but is gaining popularity among developers, as seen in the State of JavaScript survey [5].

3.6 Conclusion

In this chapter, I have compared the three most popular frameworks: React, Vue, and Svelte. The table 3.5 summarizes the key aspects analyzed in this chapter, including performance, developer experience, ecosystem, tooling, and community. For a visual comparison of the frameworks, see figure 3.16. Those results are based on the data from the previous sections and subjectively approximated to fit a scale from 1 to 5.

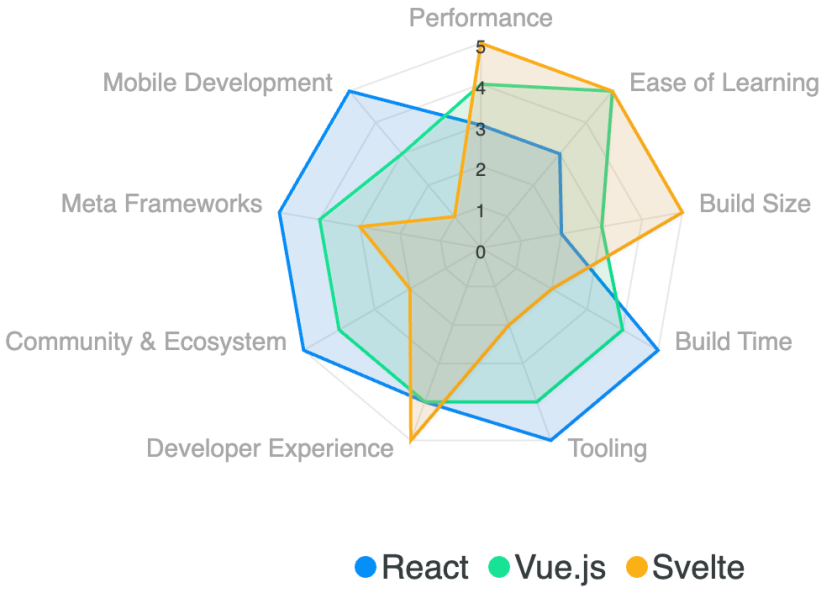


Figure 3.16: Comparison of the frameworks.

Table 3.5: Summary of the comparison of the frameworks.

	React	**Vue.js**	**Svelte**
`Performance`	Moderate, relies on Virtual DOM, slow in large DOM updates	Better optimized Virtual DOM, but higher memory usage	Fastest due to compilation-based approach, no Virtual DOM
`State Management`	Requires third-party libraries (Redux, Zustand, Context API)	Officially supported Pinia, easy to use	Built-in (Stores), most intuitive reactivity
`Event Handling`	Standard event listeners for each element	Standard event listeners for each element	Event delegation by default, reducing memory overhead

Continued on next page

Table 3.5 (continued)

	React	**Vue.js**	**Svelte**
`DOM Updates`	Mostly slowest	Mostly 2nd slowest	Fastest
`Ease of Learning`	Medium, requires an understanding of JSX and hooks	Easy, simple syntax and functions	Easy, simple syntax and functions
`Build Size`	Largest (compressed 87kB)	Moderate (compressed 55kB)	Smallest (compressed 25kB)
`Build Time`	Fastest	Fast	Slowest due to compilation
`Tooling`	Rich ecosystem, many libraries and tools	Good tooling, more built-in features	Smallest ecosystem and tooling
`Developer Experience`	Good, but requires additional libraries	Good balance between features and simplicity	Very good, straightforward
`Community & Ecosystem`	Largest, backed by Meta	Medium, backed by the Vue core team	Smallest, backed by Vercel
`Meta Frameworks`	Next.js, Remix, React Native, etc.	Nuxt.js, Quasar	SvelteKit (official)
`Animation Support`	Requires third-party libraries (Framer Motion, React Spring)	Built-in transitions but needs extra styling	Best built-in animations support
`Mobile Development`	React Native, Expo	Quasar for hybrid apps	Not supported by the framework

Final Thoughts: Which Framework is Best?

There is no single *"best"* framework—only the best choice for your specific project. Each framework excels in different areas, and the decision should

be based on your project's unique requirements. Below is a summary to guide your choice:

- **React:** If stability and ecosystem maturity are your priorities, React is the safest choice. Its extensive community and rich ecosystem make it ideal for large-scale, enterprise-level applications.
- **Vue.js:** If ease of learning and developer productivity are key, Vue.js offers the best balance. Its intuitive syntax and official libraries make it a great choice for medium to large-scale projects.
- **Svelte:** If performance and simplicity matter most, Svelte is the fastest and most lightweight framework. It is particularly suited for small to medium-sized applications where bundle size and load times are critical.

Each framework has its strengths and trade-offs. This thesis provides the necessary insights to make an informed architectural decision tailored to your project's needs.

Chapter 4

Conclusion

In this thesis, I conducted a comprehensive analysis of three modern frontend frameworks: React, Vue.js, and Svelte. Through a structured evaluation, I examined their architectural principles, performance characteristics, ease of use, and overall developer experience. By implementing real-world benchmarks and a practical prototype, I identified the strengths and weaknesses of each framework, providing valuable insights for developers and architects seeking the best tool for their projects.

Performance Analysis One of the most crucial aspects of frontend frameworks is their performance, mainly when dealing with large-scale applications. My tests revealed that Svelte consistently outperformed React and Vue.js regarding DOM update efficiency, state mutation, and event listener scalability. This is primarily due to Svelte's compiler-based approach, which eliminates the need for a Virtual DOM and generates optimized JavaScript code at build time.

Vue.js demonstrated a better performance than React in extensive data sets scenarios, as it optimizes rendering using a compiler-informed Virtual DOM. However, I measured higher memory usage compared to React and Svelte, which could be a concern for applications with limited resources. In tests with a deeply nested component tree, React outperformed Vue.js, but Svelte maintained its lead in all scenarios.

Ease of Use and Developer Experience Developer experience plays a vital role in framework adoption and my analysis distinct advantages and trade-offs for each framework.

- **React:** With its widespread adoption and large ecosystem, React offers a wealth of resources, libraries, and community support. However, its complexity adds a learning curve, particularly for newcomers. React heavily relies on third-party libraries, which can lead to fragmentation and inconsistencies in development practices.

- **Vue.js:** Vue.js is designed with ease of use, with a template-based syntax familiar to developers with HTML and CSS backgrounds. This makes it beginner-friendly while still offering advanced features for experienced developers. However, its ecosystem and community support are less extensive than React's, which may limit enterprise-level adoption.
- **Svelte:** Svelte offers a clean and intuitive syntax, making it easy to learn and use. Its built-in reactivity and state management eliminate the need for external state management solutions. Additionally, Svelte's animations and transitions are integrated directly into the framework, further enhancing usability. However, its smaller community and ecosystem may challenge developers seeking third-party libraries or support.

Ecosystem and Community Support The ecosystem and community surrounding a framework significantly impact its long-term maintenance and growth. React remains the most widely adopted framework, with extensive documentation, libraries, and community support. While not as dominant, Vue.js has a large and active community that provides a wealth of resources and plugins. Despite growing rapidly, Svelte is nowhere near competing with React or Vue.js in terms of ecosystem maturity.

Conclusion Each framework has its strengths and weaknesses, making it essential for developers to carefully consider their project requirements and team expertise when choosing a framework.

- **React** is a solid choice for large-scale applications requiring a robust ecosystem, community support, and extensive tooling. React is the safest choice for long-term projects requiring stability. However, increased complexity and reliance on third-party libraries may pose challenges for newcomers.
- **Vue.js** is the best option for developers seeking a balance between ease of use and performance. It is the golden mean of the three frameworks in terms of performance, ease of use, and community support. Vue.js is ideal for mid-sized applications.
- **Svelte** is a great choice when performance and ease of use are top priorities. However, its smaller ecosystem and community may challenge building bigger applications.

In conclusion, choosing a frontend framework depends on project requirements, team expertise, and long-term goals. This thesis provides an analytical foundation to help developers make informed decisions when selecting a framework for their project.

Further Research

While this thesis provides a comparison of React, Vue.js, and Svelte, there remain several areas that could go into more depth, but they were out of the scope of this work.

This thesis used a Todo application prototype and performance benchmarks to compare the frameworks. However, further research could involve more in-depth case studies of complex, real-world applications. This would provide additional insights into how these frameworks perform under production-like conditions.

Another area for further research is how each framework performs over time regarding maintainability. This could include analyzing how easy it is to update dependencies, refactor code, and integrate new features as the frameworks evolve.

Lastly, one large area of research could be a comparative analysis of the meta-frameworks built on top of these frameworks. These meta-frameworks, such as Next.js for React, Nuxt.js for Vue.js, and Sveltekit for Svelte, provide additional features and optimizations that can significantly impact performance and developer experience.

By exploring these areas, future work can build upon the foundation of this thesis and provide even more valuable insights into the strengths and weaknesses of these popular frontend frameworks.

Bibliography

[1] Amazon Web Services, Inc. *What is a framework?* (2025) Accessed: Apr. 5, 2025. [Online]. URL: https://aws.amazon.com/what-is/framework/.

[2] CERN. *A short history of the web.* (2025) Accessed: Apr. 5, 2025. [Online]. URL: https://home.cern/science/computing/birth-web/short-history-web.

[3] D. Chec and Z. Nowak. "The Performance Analysis of Web Applications Based on Virtual DOM and Reactive User Interfaces." In: (2018), pp. 119–134. DOI: 10.1007/978-3-319-99617-2_8.

[4] S. Greif. *Front-end frameworks ratios over time.* (2024) Accessed: Apr. 5, 2025. [Online]. URL: https://assets.devographics.com/captures/js2024/en-US/front_end_frameworks_ratios.png.

[5] S. Greif. *State of JavaScript 2024.* (2024) Accessed: Apr. 5, 2025. [Online]. URL: https://2024.stateofjs.com/en-US/.

[6] Individual mozilla.org contributors. *MDN Web Docs.* (2025) Accessed: Feb. 4, 2025. [Online]. URL: https://developer.mozilla.org/en-US/.

[7] A. Javeed. "Performance Optimization Techniques for ReactJS". In: *2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)* (2019), pp. 1–5. URL: https://api.semanticscholar.org/CorpusID:204823323.

[8] O. Kolesnikov and G. Gennady. "MODERN STATE WEB APPLICATION STATE MANAGEMENT AND REACT CONTEXT API INCORRECT USAGE FOR IT". In: *X International scientific and practical conference «Modern Trends in the Development of Scientific Space»(February 14-16, 2024) Dresden, Germany, International Scientific Unity. 2024. 286 p.* 2024, p. 104.

[9] K. Kowalczyk and T. Szandala. "Enhancing SEO in Single-Page Web Applications in Contrast With Multi-Page Applications". In: *IEEE Access* 12 (2024), pp. 11597–11614. DOI: 10.1109/ACCESS.2024.3355740.

[10] Meta Platforms, Inc. *React*. (2025) Accessed: Jan. 7, 2025. [Online]. URL: https://react.dev/.

[11] T. A. Nguyen. "A comparative analysis of Webpack and Vite as build tools for JavaScript". In: (2024).

[12] Stack Overflow. *Stack Overflow Developer Survey 2024*. (2024) Accessed: Apr. 5, 2025. [Online]. URL: https://survey.stackoverflow.co/2024/.

[13] P. Butcher, N. John, and P. D. Ritsos. "VRIA: A Web-Based Framework for Creating Immersive Analytics Experiences". In: *IEEE Transactions on Visualization and Computer Graphics* 27 (2020), pp. 3213–3225. DOI: 10.1109/TVCG.2020.2965109.

[14] S. Chen, U. R. Thaduri, and V. K. R. Ballamudi. "Front-end development in react: an overview". In: *Engineering International* 7.2 (2019), pp. 117–126.

[15] H. Al Salmi. "Comparative CSS frameworks". In: *Multi-Knowledge Electronic Comprehensive Journal for Education and Science Publications (MECSJ)* 66 (2023).

[16] Tailwind Labs Inc. *Tailwind CSS*. (2025) Accessed: Feb. 16, 2025. [Online]. URL: https://tailwindcss.com/.

[17] VoidZero Inc., Vite Contributors. *Vite*. (2025) Accessed: Jan. 25, 2025. [Online]. URL: https://vitejs.dev/.

[18] M. Wanyoike. *History of front-end frameworks*. (2018) Accessed: Apr. 5, 2025. [Online]. URL: https://blog.logrocket.com/history-of-frontend-frameworks/.

[19] University of Washington. *A brief history of HTML*. (2025) Accessed: Apr. 5, 2025. [Online]. URL: https://www.washington.edu/accesscomputing/webd2/student/unit1/module3/html_history.html.

[20] E. You. *Vue.js*. (2025) Accessed: Jan. 11, 2025. [Online]. URL: https://vuejs.org/.

[21] *Zustand*. (2025) Accessed: Apr. 5, 2025. [Online]. URL: https://zustand.docs.pmnd.rs/.

[22] Svelte contributors. *Svelte*. (2025) Accessed: Jan. 12, 2025. [Online]. URL: https://svelte.dev/.

[23] *npm trends*. (2025) Accessed: Apr. 5, 2025. [Online]. URL: https://npmtrends.com/react.